Smart Candidate Screening System — Assessment Brief

Background and Context

Every placement season, college placement cells receive resumes and academic data from dozens of final-year students who are hoping to be shortlisted by visiting companies. The placement coordinators — often just two or three staff members — are expected to go through each student's CGPA, technical skills, project work, internship history, and certifications, and form a judgment about how "placement-ready" that student is. This is done almost entirely manually today: opening spreadsheets, cross-checking skill lists against job requirements, and mentally ranking students based on gut feel rather than any consistent method.

This approach breaks down quickly as the number of students grows. Two coordinators looking at the same dataset might reach different conclusions about who is a strong candidate, because there is no shared, written-down definition of what "strong" even means. Students with genuinely excellent profiles can get overlooked simply because their resume wasn't reviewed as carefully as someone else's. Meanwhile, companies visiting for recruitment drives expect the placement team to hand them a shortlist quickly, and there usually isn't time to do a fair, thorough manual review of every single student before that deadline arrives.

The placement team wants a tool that removes this guesswork. They don't want an AI that mysteriously decides who is good and who isn't — they want a transparent, explainable system where the rules for categorizing a student are written down clearly, so that if anyone questions why a student was placed in a particular category, the coordinator can point to the exact logic and explain it in one sentence. The tool should also let them quickly narrow down the list — for example, "show me everyone with CGPA above 8 who knows React" — without opening a spreadsheet and manually scanning rows.

The Problem You Are Solving

You are being asked to design and build a system that takes in a dataset of final-year students (10–15 records, containing details like Name, CGPA, Skills, Projects, Internships, and Certifications) and turns it into something a placement coordinator can actually use in the middle of a busy recruitment drive. Specifically, the system should let them:

- See the entire list of students at a glance, with their key details visible without needing to click into each one individually.
- Narrow down the list using filters — for instance, by a minimum CGPA, or by a specific skill the visiting company is looking for.
- Instantly see which students fall into three categories — Strong, Average, and Needs Improvement — without the coordinator having to make that judgment call themselves for every student.

- Get a quick top-level summary — how many students are in the dataset in total, and how many fall into each of the three categories — so that even before scrolling through anyone's individual profile, the coordinator has a sense of the overall quality of the batch.

The most important and most evaluated part of this assignment is the categorization logic. There is no single official "correct" formula for what makes someone a Strong candidate versus someone who Needs Improvement — that is intentional. You are expected to think about it the way a placement coordinator would, decide for yourself what combination of CGPA, skill count, number of projects, internship experience, and certifications should push someone into which bucket, and then write that logic down in plain, simple language so that anyone reading your documentation — technical or not — understands exactly why a given student ended up in a given category. A system that categorizes people using logic nobody can explain is not useful to a placement team that has to defend its shortlist to visiting companies, students, or faculty.

This is not a UI design exercise. Nobody is expecting a beautifully polished interface. What matters is that the underlying structure is sound: the data is organized sensibly, the filtering genuinely works, the categorization logic is consistent and explainable, and the summary numbers on the dashboard are always accurate and update correctly as the underlying data changes.

You are also expected to treat this as a real, evolving piece of software rather than a one-shot script. Build a first working version that does the basics correctly, then go back and improve it — tighten the logic, handle edge cases in the data (a student with no listed skills, or a blank internship field), make the filtering more flexible, or make the categorization more nuanced. Your thinking process, not just the final polished output, is part of what is being evaluated.

Timeline

You have 48 hours from the point of receiving this brief and the dataset to deliver a complete, working, deployed version of the system along with full documentation.

What You Are Given

A CSV file containing details of 10–15 final-year students. Each row represents one student and includes fields such as their name, CGPA, a list of skills they know, the number or names of projects they've completed, any internship experience they have, and any certifications they hold. This is the raw data your system needs to ingest, store, and work with. You should not assume the data is perfectly clean — think about how your system should behave if a field is missing, oddly formatted, or inconsistent, the same way real-world student data usually is.

What You Must Submit at the End

By the deadline, you are expected to hand over a single, self-contained package that anyone — technical or not — can open and evaluate without needing to ask you follow-up questions. Specifically, this means:

1. **A working, deployed application** hosted on any hosting platform of your choice, accessible through a live link that can be opened directly in a browser without any local setup.
2. **A GitHub repository** containing your complete source code, structured clearly, with a proper commit history that shows your work evolving over time rather than a single final upload. The repository must also include your environment configuration file pushed directly into the repo (not excluded or gitignored), with the actual working credentials included, so that the codebase can be reviewed, run, and verified end-to-end without needing anything requested separately from you.
3. **Full documentation**, written clearly enough for a non-technical reader to follow, that includes:
    - An explanation of the problem and your understanding of it, in your own words.
    - A plain-language explanation of your categorization logic — what makes a student Strong, Average, or Needs Improvement — written so that a placement coordinator with no technical background could read it and immediately understand and trust it.
    - Screenshots covering every meaningful screen and state of the application — the full student list view, the filtering in action, the dashboard with category counts, and any individual student detail view if one exists — so that the reviewer can see exactly what the working product looks like without needing to open the live link themselves.
    - The live deployed link and the GitHub repository link, clearly labeled at the top of the document.
    - A short note on what you would improve if you had more time, showing that you're thinking about this as a real, evolving product rather than a one-off exercise.

The end goal is that a reviewer should be able to open your documentation, understand the problem, see your logic explained in plain words, look at the screenshots to understand what was actually built, click through to the live link to try it themselves, and open the GitHub repo to see the code and history — all without a single message back and forth with you.

CSV template That must be use is here :

ID,Name,Email,Phone,Branch,CGPA,Skills,Projects,Internships,Certifications
1,Aarav Gupta,aarav.g@gmail.com,9876543210,Computer Science,9.3,"MERN, AWS, Next.js","E-commerce Web App, Chat Application",SDE Intern at Amazon,"AWS Cloud Practitioner, Meta Front-End Developer"
2,Ishita Verma,ishita.v21@gmail.com,8765432109,Information Technology,8.9,"React, Node.js, MongoDB",Task Management System,Frontend Intern at TCS,Google UX Design

3,Rohan Nair,rnair.dev@gmail.com,7654321098,Computer Science,9.1,"Python, Django, React","Social Media Dashboard, Portfolio App",Web Dev Intern at Infosys,IBM Full Stack Software Developer
4,Megha Sharma,megha.sharma99@gmail.com,6543210987,Software Engineering,8.5,"Java, Spring Boot, React",Inventory Management System,Java Developer Intern at Wipro,Oracle Certified Associate
5,Karthik Reddy,karthik.r@gmail.com,9012345678,Computer Science,8.2,"JavaScript, Express, SQL","Weather App, Blog Platform",Software Intern at Tech Mahindra,HackerRank JavaScript (Basic)
6,Sneha Desai,snehad.22@gmail.com,8123456790,Electronics,7.6,"HTML, CSS, JavaScript",Personal Website,None,None
7,Aditya Kumar,aditya.k45@gmail.com,9123456781,Mechanical,7.1,"Python, SQL",Data Analysis on Sales Data,None,Google Data Analytics
8,Neha Singh,nehasingh.01@gmail.com,7123456789,Information Technology,7.8,"Java, HTML, CSS",Library Management System,Trainee at WebX,None
9,Vivek Joshi,vivek.j@gmail.com,8234567890,Computer Science,6.9,"C++, HTML",Calculator App,None,None
10,Pooja Patel,poojap.tech@gmail.com,9345678901,Civil,7.4,"Python, Excel",Student Database System,None,None
11,Amit Chawla,amit.c00@gmail.com,9456789012,Civil,5.9,MS Word,None,None,None
12,Suman Rao,suman.rao1@gmail.com,8567890123,Mechanical,6.1,"Windows, Data Entry",Basic HTML Page,None,None
13,Deepak Tiwari,deepakt.77@gmail.com,7678901234,Electronics,5.4,MS Excel,None,None,None
14,Meera Reddy,meera.r3@gmail.com,9789012345,Chemical,6.3,Basic Computer,None,None,None
15,Rajeev Menon,rajeev.m9@gmail.com,8890123456,Mechanical,5.2,MS Office,None,None,None

---